**MODULE 5**

# Cluster Analysis

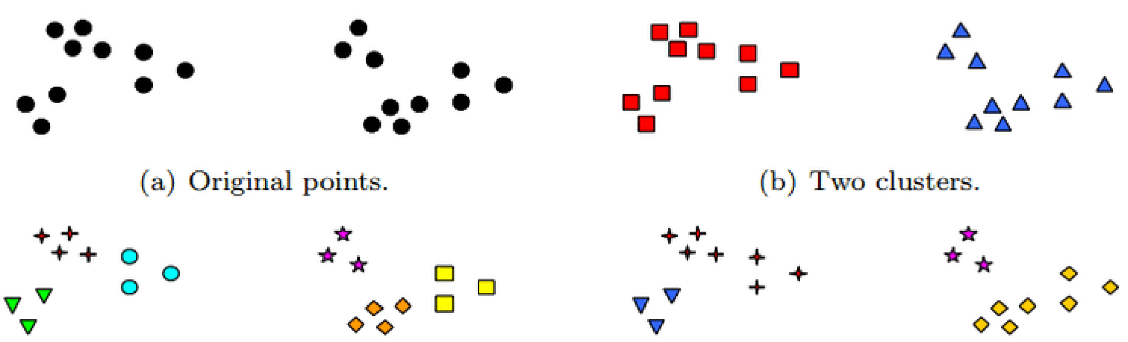
## 5.1 Introduction

Clustering is the task of dividing the population or data points into a number of groups such that data points in the same groups are more similar to other data points in the same group than those in other groups. In simple words, the aim is to segregate groups with similar traits and assign them into clusters.

**Cluster**: It is said to be "Collection of data objects"

Applications of Clustering

- ✓ Data Mining
- ✓ Pattern recognition
- ✓ Image analysis
- ✓ Bioinformatics
- ✓ Machine Learning
- ✓ Voice mining
- ✓ Image processing
- ✓ Text mining
- ✓ Web cluster engines
- ✓ Whether report analysis



(a) Original points. (b) Two clusters.

(c) Six clusters. (d) Four clusters.

## 5.2 Different types of Clustering

Hierarchical clustering Vs Partitioning clustering

Exclusive vs Overlapping Vs Fuzzy

Complete vs Partial

<u>Hierarchical clustering Vs Partitioning clustering</u>

Partitioning Clustering are clustering techniques that subdivide the data sets into a set of k groups, where k is the number of groups pre-specified by the analyst.

Hierarchical clustering is an alternative approach to partitioning clustering.

The result of hierarchical clustering is a tree-based representation of the objects

Each node(cluster) in the tree is a union of the children (sub clusters) and the root of the tree is the cluster containing all objects but leaf of the tree is single cluster of individual data objects

Exclusive vs Overlapping Vs Fuzzy

In Exclusive clustering we assign each object to a single cluster.

There are many situations in which a point could reasonably be placed in more than one cluster and these situations are better addressed by Non-Exclusive clustering

In Non-Exclusive clustering or overlapping clustering an object can simultaneously belong to more than one group(class)

Ex: A person at a university can be both an enrolled student and employee of university

Every object belongs to every cluster with membership weight that is between 0 and 1.

0- Absolutely does not belong

1- Absolutely belongs

Complete vs Partial

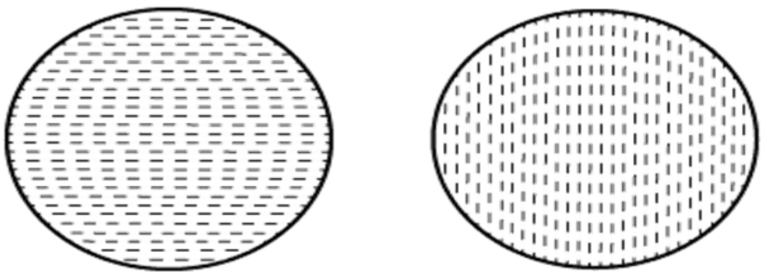
A complete clustering assigns every object to a cluster where as partial clustering does not

Example: Some newspaper stories may have a common theme, while other stories are more generic

**5.3 Types of clusters**

Well separated cluster

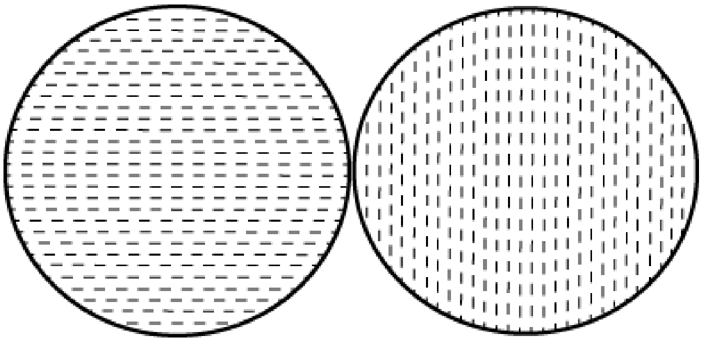
The data object within a cluster must have small distance and distance between two cluster must be high.



Well-separated clusters

Prototype based cluster (centre based cluster)

Each object is closer(similar) to the prototype that define the cluster than to the prototype of any other cluster



Center-based clusters

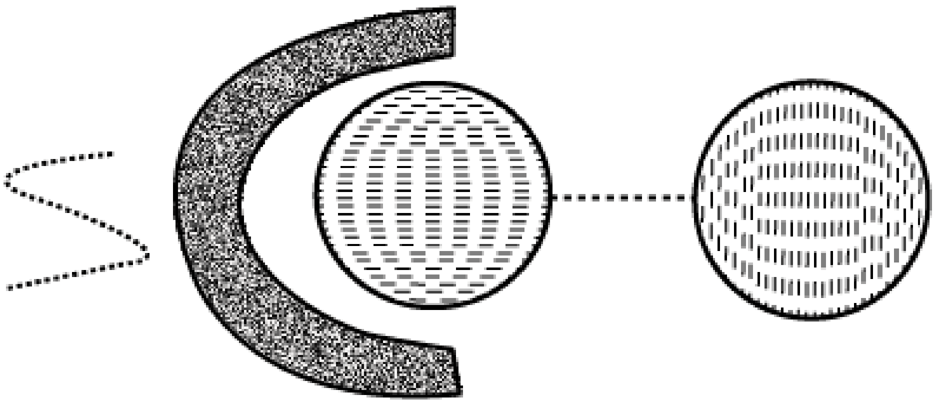
Graph based cluster (contiguity based)

Two objects are connected only if they are within a specified distance of each other.

· Each point in a cluster is closer to at least one point in the same cluster than to any point in a different cluster.

· Useful when clusters are irregular and intertwined.

· This does not work efficiently when there is noise in the data, a small bridge of points can merge two distinct clusters into one.



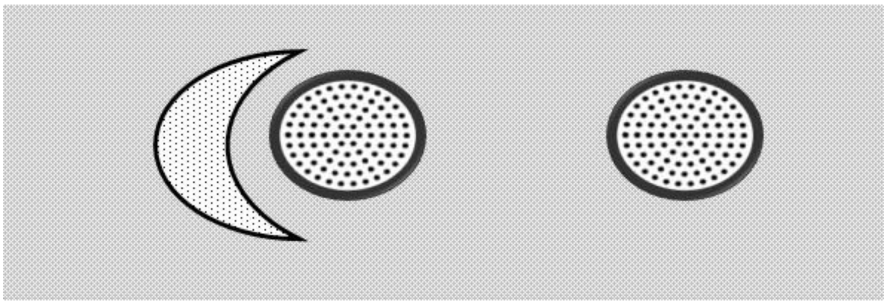
Contiguity-based clusters

Density Based Clusters:

Cluster is a dense region of objects that is surrounded by a region of low density.

Density based clusters are employed when the clusters are irregular, intertwined and when noise and outliers are present.

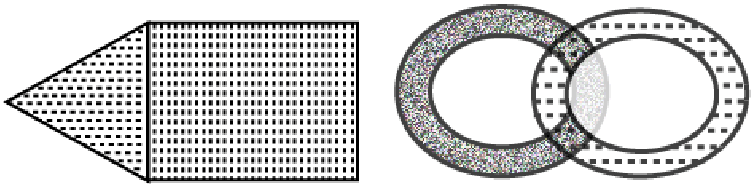
Points in low density region are classified as noise and omitted.



Shared Property cluster (Conceptual cluster)

→More general

→ Set of object that share same property



Conceptual clusters

5.4 The k-means method:

Also called the centroid method since at each step the centroid point of each cluster is assumed to be known

The k-means method uses the Euclidean distance measure

If Instead of Euclidean distance measure, the Manhattan distance is used then the method is called the k-median method

The K-means method is described as follows:

* Select the number of clusters. Let this number be k

* Pick k seed as the centroid of the k-cluster. Seed may be picked randomly

* Compute the distance of each object in the data set from each of the centroids

* Allocate each object to the cluster it is nearest to based on the distances computed in the previous step

* Compute the centroid of the cluster by computing the means of the attribute values of the object in each cluster

* Check if the stopping criteria has been met (ex. If the cluster membership is unchanged). If yes go to step7. If no go to step3.

* Stop

| STUDENT | AGE | MARK 1 | MARK 2 | MARK 3 |
|---|---|---|---|---|
| S1 | 18 | 73 | 75 | 57 |
| S2 | 18 | 79 | 85 | 75 |
| S3 | 23 | 70 | 70 | 52 |
| S4 | 20 | 55 | 55 | 55 |
| S5 | 22 | 85 | 86 | 87 |
| S6 | 19 | 91 | 90 | 89 |
| S7 | 20 | 70 | 65 | 60 |
| S8 | 21 | 53 | 56 | 59 |
| S9 | 19 | 82 | 82 | 60 |
| S10 | 47 | 75 | 76 | 77 |

Step 1: let k=3( clusters)

Step 2: let seed be the first 3 students

| STUDENT | AGE | MARK 1 | MARK 2 | MARK 3 |
|---|---|---|---|---|
| S1 | 18 | 73 | 75 | 57 |
| S2 | 18 | 79 | 85 | 75 |
| S3 | 23 | 70 | 70 | 52 |

| 1- iteration(step 3 and 4) | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **C1** | **18** | **73** | **75** | **57** | **Distance from cluster** | | | **Allocation to the nearest cluster** |
| **C2** | **18** | **79** | **85** | **75** | **C1** | **C2** | **C3** | |
| **C3** | **23** | **70** | **70** | **52** | | | | |
| S1 | 18 | 73 | 75 | 57 | 0 | 34 | 18 | C1 |
| S2 | 18 | 79 | 85 | 75 | 34 | 0 | 52 | C2 |
| S3 | 23 | 70 | 70 | 52 | 18 | 52 | 0 | C3 |
| S4 | 20 | 55 | 55 | 55 | 42 | 76 | 36 | C3 |
| S5 | 22 | 85 | 86 | 87 | 57 | 23 | 67 | C2 |
| S6 | 19 | 91 | 90 | 89 | 66 | 32 | 82 | C2 |
| S7 | 20 | 70 | 65 | 60 | 18 | 46 | 16 | C3 |
| S8 | 21 | 53 | 56 | 59 | 44 | 74 | 40 | C3 |
| S9 | 19 | 82 | 82 | 60 | 20 | 22 | 36 | C1 |
| S10 | 47 | 75 | 76 | 77 | 52 | 44 | 60 | C2 |

Step 5: Compute the centroid by computing the mean of the attribute values of the objects in each cluster

| | AGE | MARK 1 | MARK 2 | MARK 3 |
|---|---|---|---|---|
| C1 | 18.5 | 77.5 | 78.5 | 58.5 |
| C2 | 26.5 | 82.5 | 84.3 | 82 |
| C3 | 21 | 61.5 | 61.5 | 56.5 |

| 2- iteration(step 3 and 4) | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| C1 | 18.5 | 77.5 | 78.5 | 58.5 | Distance from cluster | | | Allocation to the nearest cluster |
| C2 | 26.5 | 82.5 | 84.3 | 82 | C1 | C2 | C3 | |
| C3 | 21 | 61.5 | 61.5 | 56.5 | | | | |
| S1 | 18 | 73 | 75 | 57 | 10 | 52.3 | 28 | C1 |
| S2 | 18 | 79 | 85 | 75 | 25 | 19.8 | 62 | C2 |
| S3 | 23 | 70 | 70 | 52 | 27 | 60.3 | 23 | C3 |
| S4 | 20 | 55 | 55 | 55 | 51 | 90.3 | 16 | C3 |
| S5 | 22 | 85 | 86 | 87 | 47 | 13.8 | 79 | C2 |
| S6 | 19 | 91 | 90 | 89 | 56 | 28.8 | 92 | C2 |
| S7 | 20 | 70 | 65 | 60 | 24 | 60.3 | 16 | C3 |
| S8 | 21 | 53 | 56 | 59 | 50 | 86.3 | 17 | C3 |
| S9 | 19 | 82 | 82 | 60 | 10 | 32.3 | 46 | C1 |
| S10 | 47 | 75 | 76 | 77 | 52 | 41.3 | 74 | C2 |

Note that the clusters have not changed

Cluster 1 → c1→ s1, s9

Cluster 2 → c2→s2, s5, s6, s10

Cluster 3 → c3→s3, s4, s7, s8

**Algorithm:** ***k*****-means.** The *k*-means algorithm for partitioning, where each cluster's center is represented by the mean value of the objects in the cluster.

**Input:**

- *k*: the number of clusters,
- *D*: a data set containing *n* objects.

**Output:** A set of *k* clusters.

**Method:**

(1) arbitrarily choose *k* objects from *D* as the initial cluster centers;
(2) **repeat**
(3) (re)assign each object to the cluster to which the object is the most similar, based on the mean value of the objects in the cluster;
(4) update the cluster means, that is, calculate the mean value of the objects for each cluster;
(5) **until** no change;

The *k*-means partitioning algorithm.

Strength and weakness of k-mean method:

→ k-mean is simple

→ Quite efficient even though multiple runs are often performed

→ k-mean is not suitable for all types of data

→ k-mean also have trouble in clustering data that contains outliers

→ k-mean is restricted to data for which there is a notion of centre (centroid)

**5.5 Agglomerative Hierarchical clustering**

There are two basic approaches for generating a hierarchical clustering

Agglomerative

Divisive

Agglomerative:

Start with a point as an individual cluster and at each step, merge the closest pair of clusters

Divisive:

Start with one all-inclusive cluster and at each step split a cluster until only a singleton cluster of individual parts remains

Agglomerative hierarchical clustering Algorithm:

Compute the proximity matrix, if necessary

repeat

merge the closest two clusters

update the proximity matrix to reflect the proximity between the new cluster and the original cluster

until only one cluster remains

Defining proximity between the clusters:

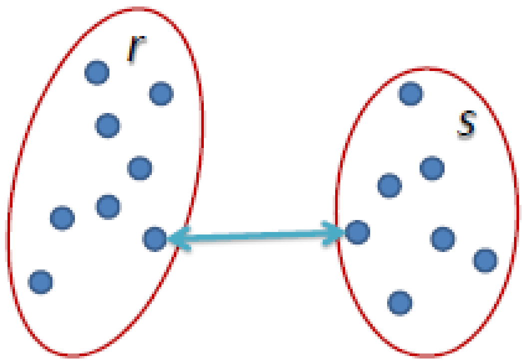
The different proximity measures in agglomerative clustering are:

MIN

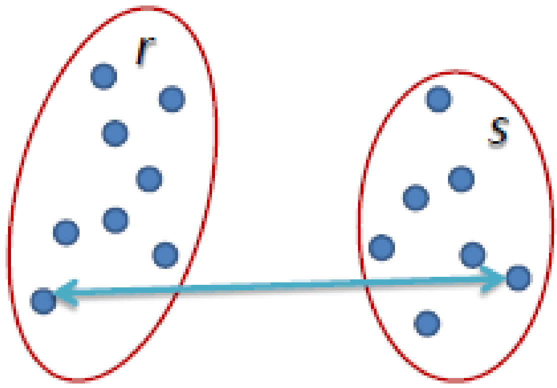
MAX

Group Average

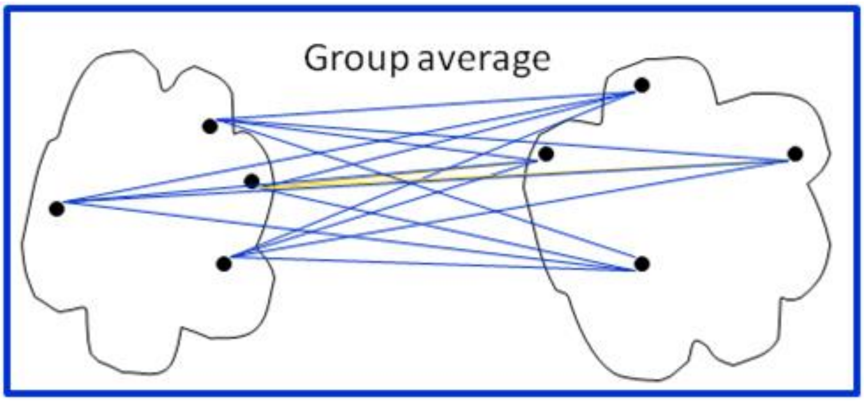
MIN defines the Proximity between the closest two points that are in different clusters




MAX defines the Proximity between the farthest two points that are in different clusters




Group Average defines the cluster proximity to be the average pairwise proximities of all pair of points from different clusters




Strengths and weaknesses

Agglomerative algorithm can produce better quality clusters

Agglomerative algorithm is expensive in terms of their computation and storage requirement

All merge can cause trouble to noise in high dimension data such as document data

**5.6 DBSCAN**

Density based clustering locates region of high dimensionality that are separated from one another by the region of low density.

DBSCAN is based on centre based approach

In centre based approach, density is estimated for a particular point in a data set by counting the number of points within a specified radius of that point

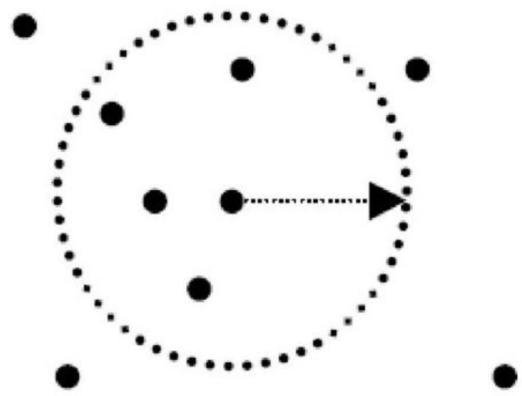


**Illustration of center-based density.**

<u>Classification of points according to centre based approaches:</u>

This approach allows us to classify the points into following categories:

Core point

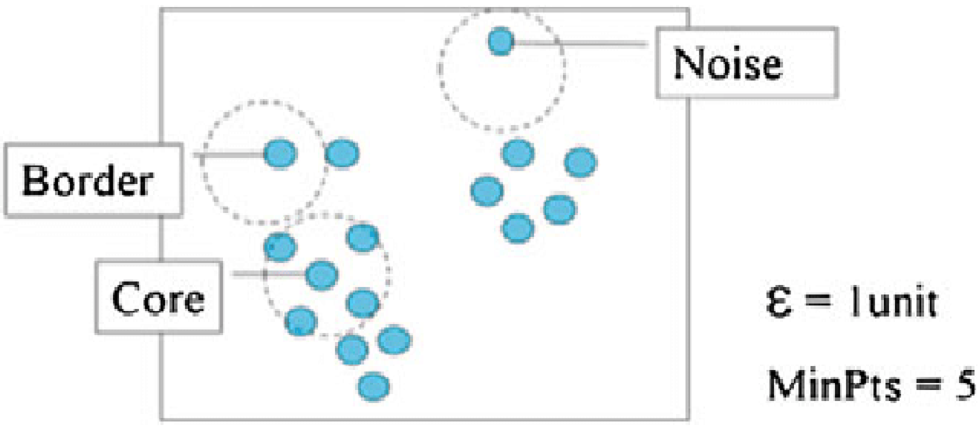
Border point

Noise or Background point

Core points are in the interior points of a density based cluster

Border points is not a core point but falls within the neighbourhood of the core point

Noise point is any point that is neither a core point nor a border point




**DBSCAN Algorithm**

* Label all the points as core, border and noise

* Eliminate noise points

* Put an edge between all the core points that are within Eps

* Make each group of connected core points to separate cluster

* Assign each border point to one of the cluster of its Associated core points

**Strength and weakness**

It is resistant to noise

Can handle cluster of arbitrary shapes and sizes

It has trouble with high dimensional data because density is more difficult to define for such data

DBSCAN can be expensive when the computation of nearest neighbor requires computing all pairwise proximities

Density based clustering

DBSCAN, a simple but effective algorithm for finding density based cluster.

Dense regions of objects that are surrounded by low density regions

Grid based cluster

Breaks the data space into grid cells and then forms cluster from cells that are sufficiently dense

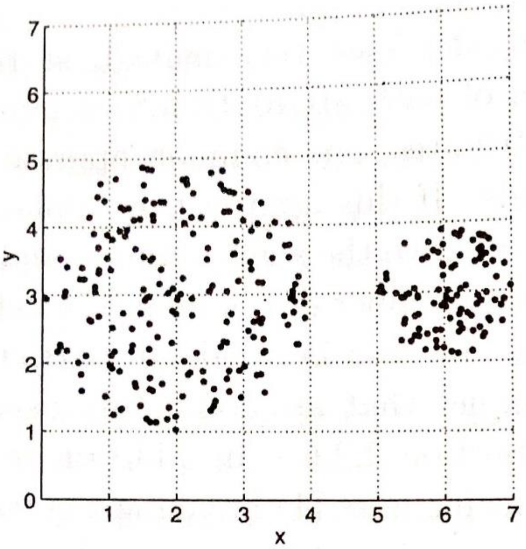



**Figure 9.10.** Grid-based density.

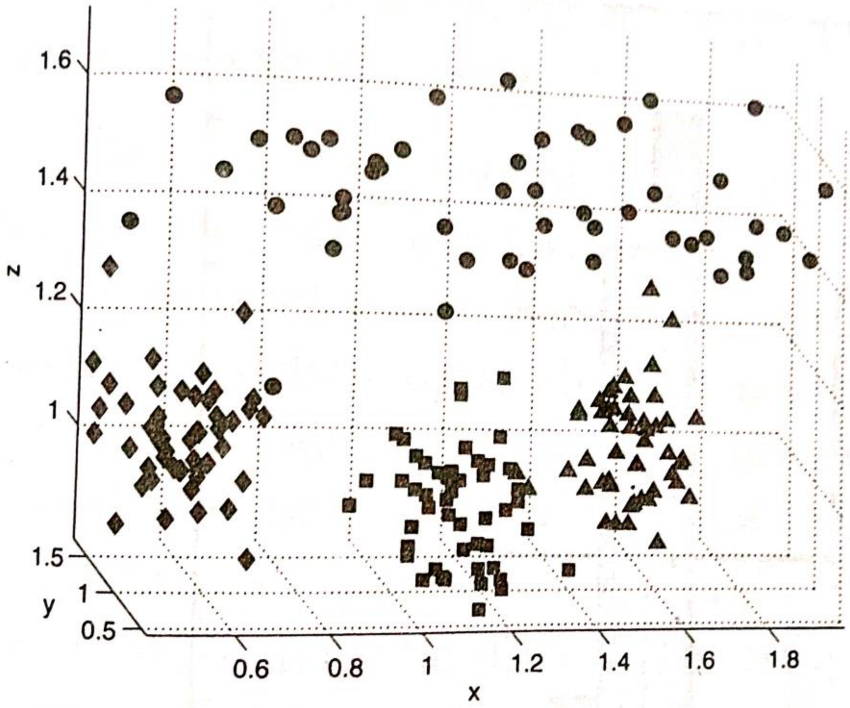
| | | | | | | |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 4 | 17 | 18 | 6 | 0 | 0 | 0 |
| 14 | 14 | 13 | 13 | 0 | 18 | 27 |
| 11 | 18 | 10 | 21 | 0 | 24 | 31 |
| 3 | 20 | 14 | 4 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

**Table 9.2.** Point counts for grid cells.

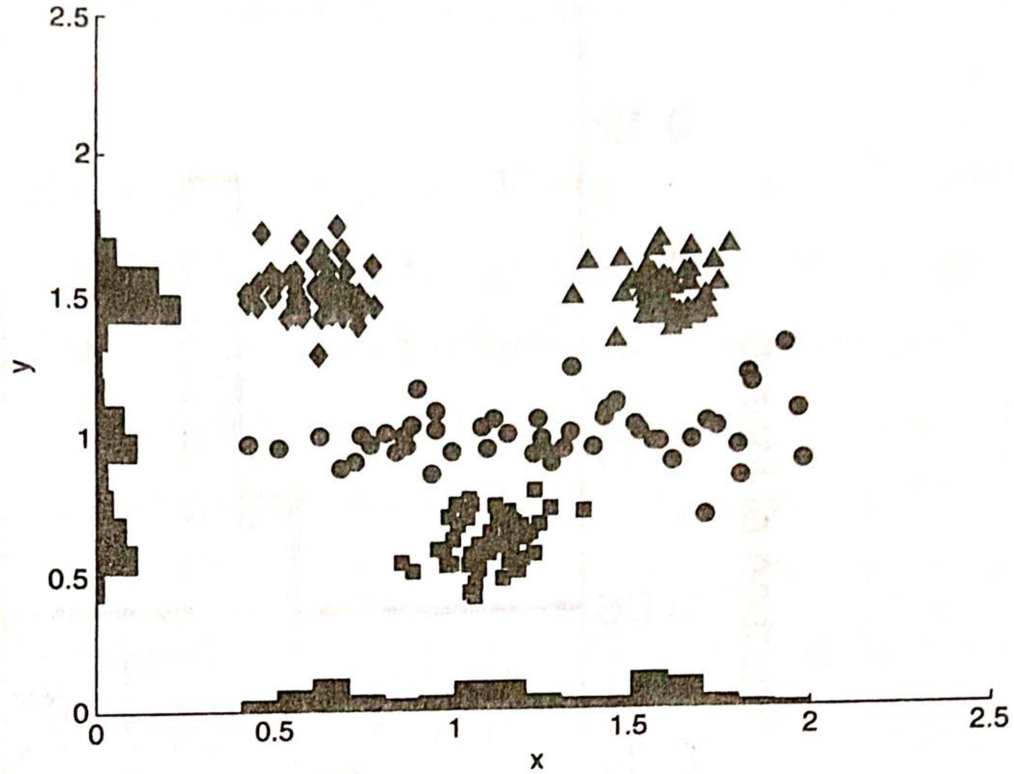
Subspace clustering

The clustering techniques considered until now found clusters by using all of the attributes

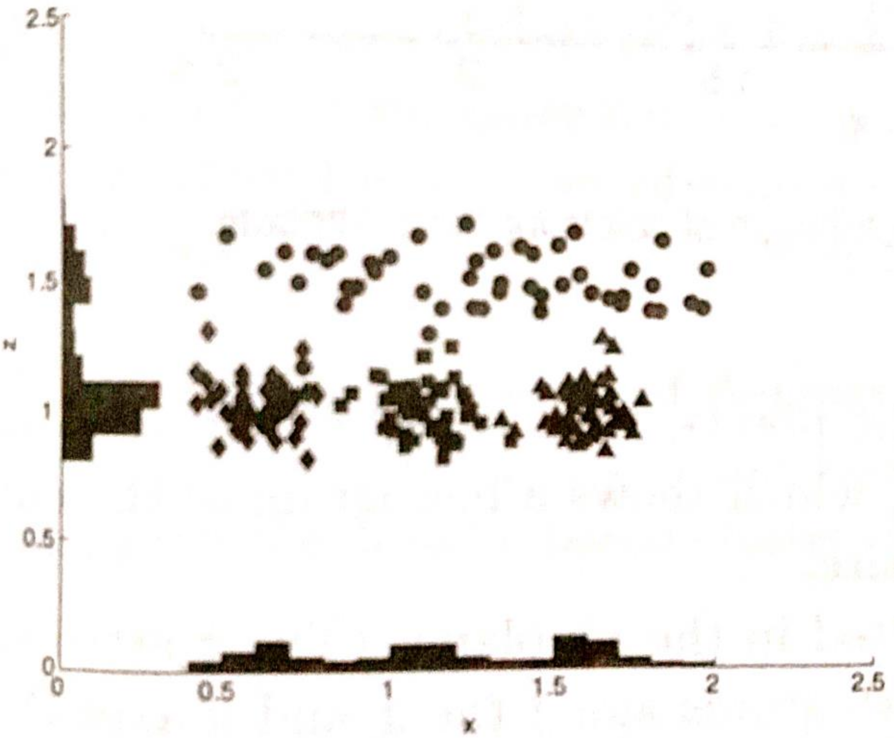
If only subset of the attributes are considered then the clusters that we find can be quite different



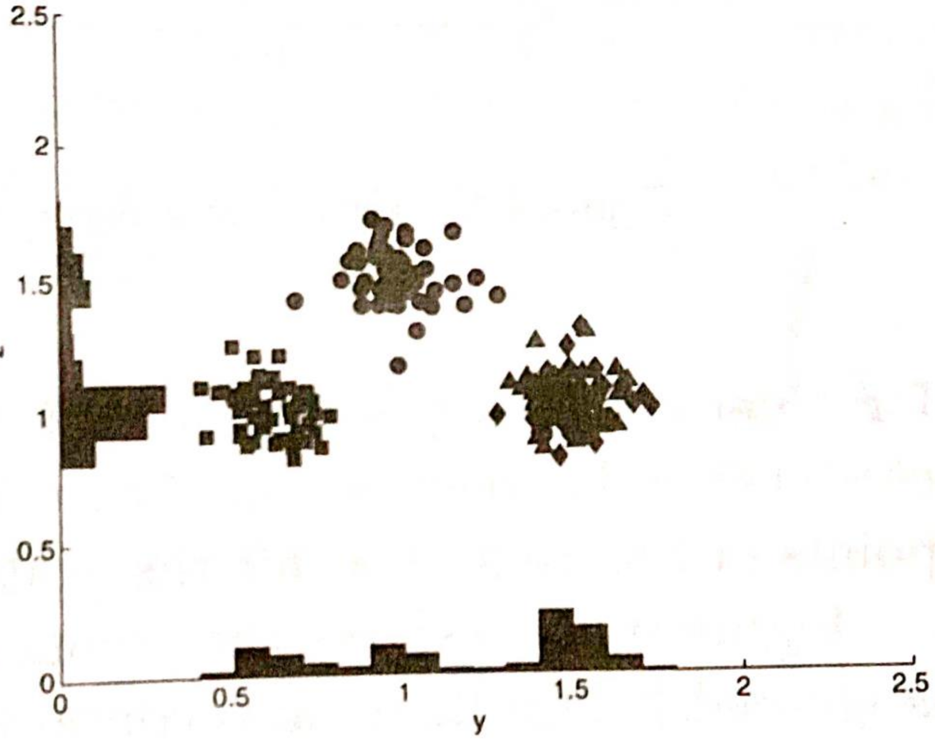

(a) Four clusters in three dimensions.




(b) View in the $xy$ plane.




(c) View in the $xy$ plane.




(d) View in the $xy$ plane.

**5.7 Cluster Evaluation**

Cluster Evaluation should be a part of any of the Cluster Analysis

Cluster evaluation/Validation are traditionally classified into following types:

Unsupervised cluster Evaluation

Supervised cluster Evaluation

Unsupervised cluster Evaluation

Measure the goodness of the clustering structure without considering external information

Unsupervised measure is further divided into two classes:

Measure of cluster cohesion

Measure of cluster separation

Cluster cohesion determines how closely related to objects in a cluster

Cluster separation determines how distinct or well separated a cluster is from other clusters

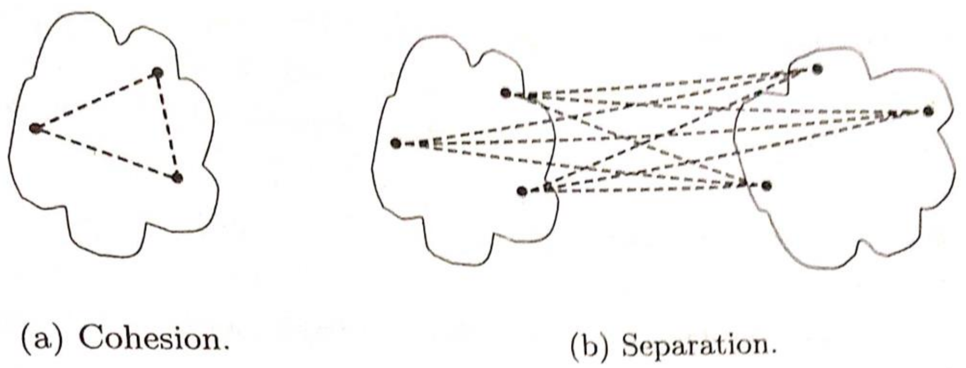


**Figure 8.27.** Graph-based view of cluster cohesion and separation.

$$cohesion(C_i) = \sum_{\substack{\mathbf{x}\in C_i \\ \mathbf{y}\in C_i}} proximity(\mathbf{x}, \mathbf{y})$$

$$separation(C_i, C_j) = \sum_{\substack{\mathbf{x}\in C_i \\ \mathbf{y}\in C_j}} proximity(\mathbf{x}, \mathbf{y})$$

Supervised cluster Evaluation

This approach measures the extent to which two objects that are in the same class are in the same cluster and vice versa

There are two types of supervised measures:

Classification oriented

Similarity oriented

Classification oriented

No. of measures such as “Entropy”, “Purity”, “Precision”, “Recall”, “F measure” are commonly used to evaluate the performance

***Entropy:***

The degree to which each cluster consists of objects of a single class

$$e = \sum_{i=1}^{K} \frac{m_i}{m} e_i,$$

where K is the number of clusters

m is the total number of data points

$$e_i = -\sum_{j=1}^{L} p_{ij} \log_2 p_{ij},$$

pij=mij/mi

where mi is the number of objects in cluster i

mij is the no of objects of class j in cluster i

L is the number of classes

***Purity***:

$$purity = \sum_{i=1}^{K} \frac{m_i}{m} p_i.$$

where pi= max pij

***Precision***:

The fraction of a cluster that consists of objects of a specified class

precision(i,j)=pij

***Recall***

The extent to which a cluster contains all the classes

***F measure***:

combination of both precision and recall

Extent to which a cluster contains only objects of a particular class and all the objects of that class

Similarity oriented

Cluster evaluation will be done using comparison of two matrices

Ideal cluster similarity matrix

Ideal class similarity matrix

Ideal cluster similarity matrix

Has 1 in ijth entry if two objects i and j are in the same cluster, and 0 otherwise

Ideal class similarity matrix

Has 1 in ijth entry if two objects i and j belong to the same class, and 0 otherwise

**Table 8.10.** Ideal cluster similarity matrix.

| Point | p1 | p2 | p3 | p4 | p5 |
|---|---|---|---|---|---|
| p1 | 1 | 1 | 1 | 0 | 0 |
| p2 | 1 | 1 | 1 | 0 | 0 |
| p3 | 1 | 1 | 1 | 0 | 0 |
| p4 | 0 | 0 | 0 | 1 | 1 |
| p5 | 0 | 0 | 0 | 1 | 1 |

**Table 8.11.** Ideal class similarity matrix.

| Point | p1 | p2 | p3 | p4 | p5 |
|---|---|---|---|---|---|
| p1 | 1 | 1 | 0 | 0 | 0 |
| p2 | 1 | 1 | 0 | 0 | 0 |
| p3 | 0 | 0 | 1 | 1 | 1 |
| p4 | 0 | 0 | 1 | 1 | 1 |
| p5 | 0 | 0 | 1 | 1 | 1 |